

University of Asia Pacific

Department of Computer Science and Engineering

CSE 402:Operating System Lab

LAB REPORT

Submitted by: Fardin Rohit

Student ID: 23101149

Section: C-2

Lab Task: 03

Submitted to:
Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 10/08/2026

1. Objective

To implement First Come First Serve (FCFS) and Shortest Job First (SJF, non-preemptive) CPU scheduling algorithms in Python, compute Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) for a given set of processes, and compare the performance of both algorithms.

2. Theory

2.1 First Come First Serve (FCFS)

FCFS is the simplest CPU scheduling algorithm. Processes are executed strictly in the order they arrive in the ready queue (i.e., sorted by Arrival Time). The CPU is allocated to a process on a non-preemptive basis, meaning once a process starts execution it runs to completion before the next process begins.

2.2 Shortest Job First (SJF) — Non-Preemptive

SJF selects, from the set of processes that have already arrived, the one with the smallest Burst Time (BT) to execute next. It is non-preemptive here, so once a process starts, it is not interrupted. SJF is provably optimal for minimizing average waiting time among non-preemptive algorithms, since running the shortest available job first reduces the time other jobs spend waiting behind long jobs.

3. Input Data (Process Set)

PID	Arrival Time (AT)	Burst Time (BT)
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

4. C++ Implementation

4.1 FCFS Function

```
void performFCFS(Process p[], int n) {
    sort(p, p + n, [](const Process &a, const Process &b) {
      if (a.at != b.at) return a.at < b.at;
      return a.pid < b.pid;
    });

    int currentTime = 0;

    for (int i = 0; i < n; i++) {
      if (currentTime < p[i].at) {
         currentTime = p[i].at;
      }

      currentTime += p[i].bt;
      p[i].ct = currentTime;
      p[i].tat = p[i].ct - p[i].at;
      p[i].wt = p[i].tat - p[i].bt;
    }
}
```

4.2 SJF (Non-Preemptive) Function

```
void performSJF(Process p[], int n) {
    bool done[n];
    fill(done, done + n, false);

    int completed = 0;
    int currentTime = 0;

      while (completed < n) {
        int idx = -1;
        int minBT = 1e9;

        for (int i = 0; i < n; i++) {
            if (!done[i] && p[i].at <=
currentTime) {
                if (p[i].bt < minBT ||
                   (p[i].bt == minBT && p[i].at
< p[idx].at)) {
                    minBT = p[i].bt;
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            int nextArrival = 1e9;
            for (int i = 0; i < n; i++) {
                if (!done[i]) nextArrival =
min(nextArrival, p[i].at);
            }
            currentTime = nextArrival;
            continue;
        }

        currentTime += p[idx].bt;
```

```
        p[idx].ct = currentTime;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;

        done[idx] = true;
        completed++;
    }
```

5. Output

5.1 FCFS Result

PID	AT	BT	CT	TAT	WT
P1	3	3	12	9	6
P2	2	5	9	7	2
P3	5	4	16	11	7
P4	1	3	4	3	0
P5	6	2	18	12	10

Average TAT = 8.40 Average WT = 5.00

5.2 SJF Result

PID	AT	BT	CT	TAT	WT
P1	3	3	7	4	1
P2	2	5	18	16	11
P3	5	4	13	8	4
P4	1	3	4	3	0
P5	6	2	9	3	1

Average TAT = 6.80 Average WT = 3.40

5.3 Comparison

Metric	FCFS	SJF
Average TAT	8.40	6.80
Average WT	5.00	3.40

6. Discussion

6.1 Why is FCFS Required in CPU Scheduling?

- **Simple:** FCFS is easy to understand and implement using a FIFO queue.
- **Fair:** Processes are executed in the order they arrive.
- **No starvation:** Every process eventually gets CPU time.
- **Low overhead:** It does not require burst-time estimation or queue reordering.
- **Baseline:** FCFS can be used to compare the performance of other scheduling algorithms.

6.2 Why Does SJF Perform Better Than FCFS?

- **Lower waiting time:** SJF executes shorter processes first, reducing the average waiting time.
- **Reduces convoy effect:** Short processes do not have to wait behind long processes.
- **Better results:** In this experiment, SJF achieved an average TAT of **6.80** and WT of **3.40**, compared with **8.40** and **5.00** for FCFS.
- **Limitation:** SJF requires knowledge of burst times and may cause starvation of longer processes.

7. Conclusion

Both FCFS and SJF were implemented and tested using the same five processes. SJF performed better in this experiment, achieving lower average turnaround time (**6.80**) and waiting time (**3.40**) than FCFS (**8.40** and **5.00**). However, FCFS remains useful because of its simplicity, fairness, and low overhead.

Github link: [CSE-402/Lab-03-SJF-Comparison-with-FCFS/report at main · 23101149-cmyk/CSE-402]